EmoSense — Deployment Documentation

From the model file to a working phone app: GitHub, Render, UptimeRobot and API tests

Contents of this document

- 03-deployment/README.md

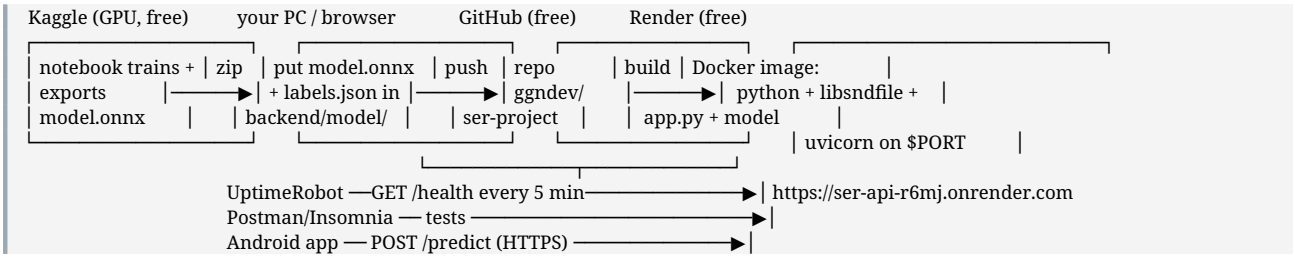
Generated from the Markdown files in the docs folder (the Markdown files remain the source of truth).

3. Deployment — from a model file to a working phone app

Purpose

Explain **how the trained model reached the phone** and *why each step and each service was chosen*. The constraint that shaped everything: **the whole system had to be free and need no credit card**.

The journey in one picture



Why each choice

Step / service	What it does	Why chosen	What we rejected (and why)
Kaggle notebook	Free GPU for training (≈ 30 h/week)	Free GPU, datasets already hosted there	Colab (quota ran out), training on the laptop (no GPU, 7 GB RAM)
ONNX file	The single artefact that is deployed	Small runtime, no PyTorch	Shipping the notebook/PyTorch
GitHub repository	Stores the code **and** the model	Render deploys straight from GitHub; every push redeploys	Uploading files by hand to the server
Render web service (free, Docker)	Builds the image and runs `uvicorn` with HTTPS	Free, **no card**, HTTPS included, auto-deploy on push	Hugging Face Docker Spaces (now paid), Heroku (no free tier), Google Cloud Run (needs a card), Fly.io (needs a card)
Docker	Pins Python, system libraries and packages	"Works on my machine" = "works on the server"	Native Python runtime: no control over `libsndfile`
Model inside the repo	`backend/model/model.onnx` (≈ 95 MB int8)	Simple: the build needs nothing else. GitHub accepts files up to 100 MB via git	Release download (kept as a fallback in the Dockerfile)
int8 Wav2Vec2 model	The deployed network	378 MB float32 did not fit the 512 MB free instance; int8 ≈ 95 MB and ≈ 340 MB RAM at run time	CNN-BiLSTM (8 MB) is the further fallback if memory runs out
UptimeRobot	Pings `/health` every 5 min	Free instances sleep after 15 minutes idle (≈ 1 min cold start); regular pings keep it awake	Paying for an always-on instance

Step / service	What it does	Why chosen	What we rejected (and why)
Postman / Insomnia collection	Automated API tests	Proves the API behaves (including error cases) before the app depends on it	Testing only through the app

Step by step (what was actually done)

Step 1 – Get the model out of the notebook

The notebook's final cell writes `model.onnx` (float32), `model_int8.onnx` and `labels.json`. Download them from Kaggle's *Output* tab. `labels.json` must come **with** the model because it records the class order and preprocessing. *Why both files?* The API loads `MODEL_FILE` (default `model.onnx`); to serve the int8 model we copied `model_int8.onnx` over `backend/model/model.onnx`.

Step 2 – Put the project on GitHub

The repository `ggndev/ser-project` contains `backend/`, `android/` and the documentation. `backend/model/model.onnx` is below GitHub's 100 MB per-file limit, so ordinary `git push` works. *Why GitHub?* Render connects to it and **rebuilds automatically on every push** – the deployment pipeline is just "push". *Pitfall we met:* GitHub's website refuses single files over 25 MB, so a 95 MB model cannot be uploaded in the browser; use `git push` (or a Release asset, which the Dockerfile can download).

Step 3 – Create the Render service

Setting	Value	Why
Source	GitHub repo `ggndev/ser-project`, branch `main`	auto-deploy from the repo
Language	**Docker**	use our `Dockerfile`
Root Directory	`backend`	the Dockerfile is in `backend/`, not in the repo root. *Our first deploy failed with "failed to read dockerfile: open Dockerfile: no such file or directory" because this was empty.*
Dockerfile path	`./Dockerfile`	relative to the root directory
Instance type	**Free**	no cost, no card
Health check path	`/health`	Render restarts the service if it stops answering

Render then: clones the repo → builds the image (installs `libsndfile1`, then the Python packages, copies the code and model) → starts `uvicorn` on the port Render provides → serves it at `https://ser-api-r6mj.onrender.com`.

Step 4 – Verify

1. Open `https://ser-api-r6mj.onrender.com/health` → expect `{"status":"ok","model":"model.onnx","labels":[...]}`.

2. Open /docs → FastAPI's interactive page, where you can upload a WAV and press *Execute* (a quick demo for the mentor).
3. Run the Postman/Insomnia collection (below).

Step 5 – Keep it awake

Create a free monitor at uptimerobot.com: *HTTP(s)*, URL …/health, interval 5 minutes. Free Render services spin down after 15 minutes without traffic; the first request after sleeping needs ≈ 1 minute. The monitor's ping counts as traffic. *Free-tier arithmetic:* Render gives 750 instance-hours per month ≈ 31 days × 24 h, i.e. exactly one always-on free service.

The test collection (Postman / Insomnia)

File: postman/SER_API.postman_collection.json (13 requests). Import it in Postman (Insomnia imports Postman collections too; it ignores the test scripts, so check the status codes by eye). Set Postman's *working directory* to the postman folder so the sample audio in postman/samples/ is found.

#	Request	Expected	What it proves
01	GET /health	200, status: ok, 7 labels	service alive, model loaded
02	GET /	200, labels	root info
03 – 07	POST /predict with a happy/sad/surprise/neutral/fear clip	200, valid emotion, confidence 0–1, probabilities sum to 1	the whole pipeline works on real audio
08	POST /predict with a 9 s clip	200, 2–4 windows	multi-window logic (this test exposed the memory crash that led to one-window-per-call)
09	POST /predict with silence	200, speech_detected: false, emotion: null	the silence gate works
10	POST /predict with a text file	400 "could not decode audio"	bad input handled, not a server crash
11	POST /predict without a file	422	FastAPI validation
12	GET /predict	405	wrong HTTP method rejected
13	GET /nope	404	unknown route

Lines such as pm.test("200 OK", () => pm.response.to.have.status(200)); inside the collection are the automated assertions Postman runs after each request.

What each HTTP status means (useful in the viva)

Code	Meaning	Where it occurs here
200	OK	successful prediction or health check
400	Bad request	empty or undecodable file
404	Not found	unknown URL
405	Method not allowed	GET on /predict
422	Unprocessable	missing file field

Code	Meaning	Where it occurs here
502	Bad gateway	Render's proxy could not get an answer (we saw it when the server ran out of memory)

Updating the system later

Change	What to do
New code in `app.py`	upload/push it to GitHub → Render redeploys automatically (~3–5 min)
New model	replace `backend/model/model.onnx` and `labels.json` in the repo (use `git push` because of the size)
Switch to the small CNN	copy `backend/model_cnn/model.onnx` and `labels.json` over `backend/model/`
New server address in the app	⚙️ Settings screen in the app (no rebuild needed)

Limits to state honestly

- **Latency:** the free instance has a fraction of a CPU: expect several seconds per 3-second window, so a 15-second recording takes up to ≈ 35 s. The app shows an "Analysing…" state and waits up to 90 s.
- **Cold start:** up to ≈ 1 minute after idle (UptimeRobot reduces this).
- **Open API:** no authentication on `/predict` — acceptable for a demo; the app's login only guards the app screen.
- **Ephemeral disk:** nothing is stored on the server (audio is processed in memory), which is also good for privacy.

Things to tell the mentor

- The **constraints (free, no card, 512 MB)** drove every decision: ONNX instead of PyTorch, int8 quantisation, a slim Docker image, one window per model call.
- **Test-driven deployment:** the Postman collection found a real defect (memory crash on multi-window clips) before the app depended on the API.
- The design is **portable**: the Docker image runs the same on any host that supports Docker.